

Design Rational

Design Decisions/Chosen Patterns

Currently we are implementing the **factory** and **singleton** patterns. We Factory is used because several classes will have many objects created at different times during gameplay. For example, different crop types (wheat, tomato, corn) and animals (chicken, cow) are created by the Market and when harvested. The **Singleton** pattern is used for the **Menu** and **Shop** because each should exist as a single instance for the lifetime of the game.

Our structural design pattern is the **facade** pattern. Our game and UI are run through a Game class. This class hides the many interfaces and classes that hold the implementations of the character, their actions, the objects that can be interacted with, the market and its actions, etc. The Game class shows a simple UI and game experience and shows none of the complex logic to the player.

At this point we have read ahead a little bit and have implemented a behavioural pattern: **State**. This pattern changes an object's behavior depending on the state: that the object is in. We use this for the ICrop-derived classes, the IAnimal-derived classes, and the ITile-derived classes. For ICrops and IAnimal classes, the objects change state when they are fed or watered. As well if they are ready to be harvestable. The ITile classes have a state for if they are watered. Watering the Field class allows for the crops on it to grow, but only if the state of the Field is watered.

Changes from deliverable 1

One of the big changes we made from deliverable 1 is the creation of the TileBoard class. This class acts as a factory/build system for the world grid, creating the game environment, or board, that the player walks around on. We did this because we need the entire walkable space to be interactable, so to cover the

game window in QT Creator we needed to make a matrix of our ITile classes. We thought this building of the game map best to be in its own class.

In deliverable 1 we failed to mention a class that would run the game. We have now made that class and added it to all the diagrams. This game class owns all the key subsystems such as Player, TileBoard, Market, and Menu, and manages them through a QT event loop (a function used to keep track of time and make the game loop). This class is where the main window of the game is created and where mouse and keyboard inputs are logged and sent to the appropriate class to cause an action.

We are also creating a new class called inventory. We originally had the inventory as an array/vector or a dictionary, but ran into issues with trying to store multiple types in the same data structure (as C++ is a strong type language). Therefore we are making an inventory class so that we could store multiple object types in one container.

We have also decided to split the Character class in two. We now have a Player class and a Character class. The Player class will act as the player in and will be called by the Game class. It will hold an instance to the Inventory and the Character classes. The character class will handle the movement and the rendering of the sprites for the Player class.

Buttons for game

- Movement WASD
- Interaction LEFT MOUSE CLICK or E
- Main menu ESC
- Market M (Toggle between buy and sell mode TAB)
- Go forward a day T

Next Steps

Currently we have the full game cycle working for crops, being able to buy, plant, grow, and sell them. The implementation of our animal classes will be the final step in creating the game logic. In terms of design patterns we need to further implement them. We have the facade pattern, singleton pattern, and state pattern in use, but we still need to properly implement the factory pattern. Currently our factory has two uses, creating the object (which is the main purpose) but it also creates the map to be used. This violated the Single Responsibility Principle. We also need to add a factory for the other interfaces (ICrops and IAnimal). Other things that we could add if time allows are decorations for the map and other UI elements. This would include non-interactable objects such as trees or fences for a more engaging visual experience, as well as a cleaner look for the Menu and Market classes. Finally we want to try to refactor our movement and interactions to follow the command design pattern lowering the coupling and adding more scalability for our future features of the game.

With the IAnimal classes we need to implement the remaining animal gameplay loop: Placing animals on the boards via player inventory, and making the animals interactable (feed) and harvestable for their produce (Egg, Milk). Persist and restore game state -> Develop the Save/Load menu (Singleton) with a serialized format (JSON/txt) to allow gameplay over multiple sessions.

Youtube Link: <https://www.youtube.com/watch?v=3qII09maDNU>

[illegible]

